Anomaly Detection & Time Series

Assignment

Question 1: What is Anomaly Detection? Explain its types (point, contextual, and collective anomalies) with examples.

Anomaly Detection is a machine learning and data analysis technique used to identify data points, events, or observations that significantly deviate from the normal behavior of a dataset. These unusual observations, known as anomalies or outliers, may indicate important events such as fraud, network intrusions, equipment failures, medical abnormalities, or data errors. Anomaly detection can be performed using statistical methods, machine learning algorithms, or deep learning techniques. There are three main types of anomalies. **Point anomalies** occur when a single data instance is significantly different from the rest of the data; for example, a credit card transaction of ₹1,00,000 made by a user who usually spends less than ₹5,000. **Contextual anomalies** occur when a data instance is abnormal only within a specific context, such as a temperature of 35°C being normal in summer but unusual in winter. Here, the anomaly depends on contextual information like time or location. **Collective anomalies** occur when a group of related data instances behaves abnormally even though individual points may appear normal; for example, a sudden sequence of unusual network requests indicating a cyberattack. Detecting these anomalies is crucial in various domains because it helps organizations identify risks, prevent losses, improve security, and maintain system reliability. The choice of anomaly detection method depends on the type of anomaly and the characteristics of the dataset.

Question 2: Compare Isolation Forest, DBSCAN, and Local Outlier Factor in terms of their approach and suitable use cases.

Isolation Forest, DBSCAN, and Local Outlier Factor (LOF) are popular anomaly detection techniques, but they identify outliers using different approaches and are suitable for different scenarios. **Isolation Forest** is an ensemble-based algorithm that detects anomalies by randomly partitioning data using decision trees. Since anomalies are rare and different from normal observations, they tend to be isolated in fewer splits, resulting in shorter path lengths. It is highly efficient for large, high-dimensional datasets and is commonly used in fraud detection, network security, and financial analysis. **DBSCAN (Density-Based Spatial Clustering of Applications with Noise)** is a density-based clustering algorithm that groups closely packed data points and labels points in low-density regions as noise or anomalies. It works well for datasets with irregularly shaped clusters and is useful in spatial data analysis, image processing, and geographical applications. However, its performance depends heavily on the selection of parameters such as `eps` and `min_samples`. **Local Outlier Factor (LOF)** is also a density-based method, but instead of identifying global outliers, it measures the local density of a data point relative to its neighbors. A point is considered anomalous if its density is significantly lower than that of surrounding points. LOF is particularly effective when datasets contain clusters of varying densities and is often used in intrusion detection and customer behavior analysis. While Isolation Forest is scalable and efficient, DBSCAN

excels at detecting noise in clustered data, and LOF is best for identifying local anomalies in complex datasets.

Question 3: What are the key components of a Time Series? Explain each with one example.

A Time Series is a sequence of observations recorded over time at regular intervals, and it is commonly used in forecasting, trend analysis, and decision-making. The key components of a time series are **Trend, Seasonality, Cyclical Variation, and Irregular (Random) Variation**. **Trend** represents the long-term direction of the data, whether increasing, decreasing, or stable over time. For example, the annual growth in online shopping sales over the last decade shows an upward trend. **Seasonality** refers to regular and predictable patterns that repeat at fixed intervals, such as daily, monthly, quarterly, or yearly. For instance, ice cream sales typically increase during summer and decrease during winter every year. **Cyclical Variation** consists of fluctuations that occur over longer periods and are usually associated with economic or business cycles. Unlike seasonality, these cycles do not have a fixed duration. For example, automobile sales may rise during periods of economic growth and decline during recessions. **Irregular or Random Variation** includes unpredictable fluctuations caused by unexpected events such as natural disasters, strikes, pandemics, or sudden market changes. For example, airline passenger traffic dropped sharply during the COVID-19 pandemic, creating an irregular variation in the data. Understanding these components helps analysts decompose time series data, identify underlying patterns, and build accurate forecasting models for applications such as sales prediction, stock market analysis, weather forecasting, and demand planning.

Question 4: Define Stationary in time series. How can you test and transform a non-stationary series into a stationary one?

Stationarity is a fundamental concept in time series analysis where the statistical properties of a series, such as **mean, variance, and autocorrelation**, remain constant over time. In a stationary time series, the data fluctuates around a fixed mean with constant variability, making it easier to model and forecast. Many forecasting techniques, such as ARIMA, assume that the underlying data is stationary. A non-stationary series may exhibit trends, seasonality, changing variance, or structural changes over time. For example, the monthly sales of a growing company often show an upward trend and are therefore non-stationary. To test whether a series is stationary, analysts use visual inspection through time series plots and statistical tests such as the **Augmented Dickey-Fuller (ADF) Test** and the **KPSS Test**. In the ADF test, a low p-value (typically less than 0.05) indicates that the series is stationary. If a series is non-stationary, it can be transformed into a stationary one using several techniques. **Differencing** subtracts consecutive observations to remove trends, **seasonal differencing** removes seasonal patterns, **logarithmic or square root transformations** stabilize variance, and **detrending** removes long-term trends from the data. For example, taking the difference between consecutive monthly sales values can eliminate a growth trend and produce a stationary series. Converting data into a stationary form improves the accuracy and reliability of time series models and forecasting results.

Question 5: Differentiate between AR, MA, ARIMA, SARIMA, and SARIMAX models in terms of structure and application.

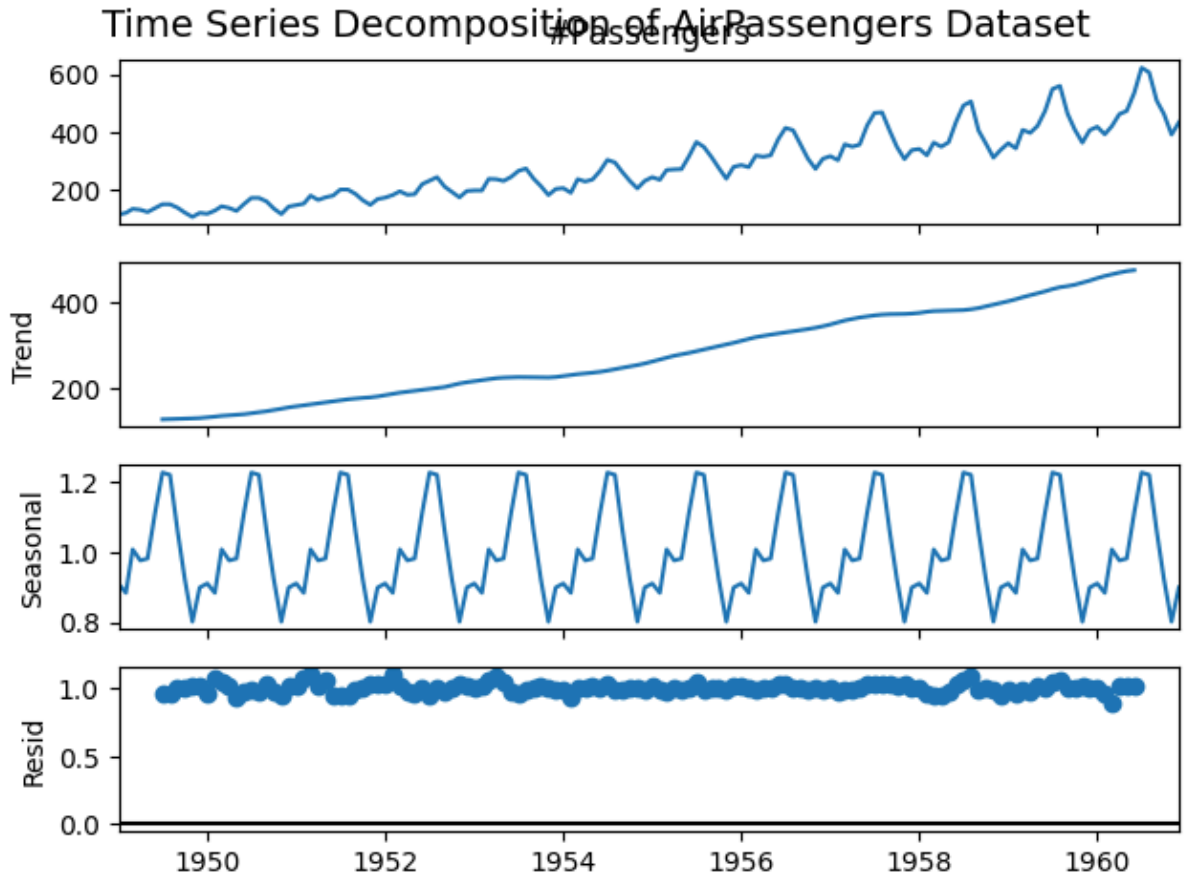
AR, MA, ARIMA, SARIMA, and SARIMAX are widely used time series forecasting models that differ in their structure and ability to handle trends, seasonality, and external variables. The **AutoRegressive (AR)** model predicts future values using a linear combination of past observations, making it suitable for stationary data with temporal dependence. For example, today's stock price may be predicted using prices from previous days. The **Moving Average (MA)** model predicts values based on past forecast errors rather than past observations and is useful when random shocks influence the series. **ARIMA (AutoRegressive Integrated Moving Average)** combines AR and MA components and includes an **Integrated (I)** component, which uses differencing to make non-stationary data stationary. ARIMA is widely used for forecasting data with trends but without seasonality. **SARIMA (Seasonal ARIMA)** extends ARIMA by adding seasonal autoregressive, differencing, and moving average terms, making it suitable for data with repeating seasonal patterns, such as monthly sales that peak every holiday season. **SARIMAX (Seasonal ARIMA with eXogenous variables)** further extends SARIMA by incorporating external or explanatory variables that may influence the target series. For example, sales forecasting may use advertising expenditure, weather conditions, or economic indicators as additional inputs. In summary, AR and MA handle simple stationary series, ARIMA handles non-stationary series with trends, SARIMA handles seasonal time series, and SARIMAX is best when both seasonality and external influencing factors need to be considered for accurate forecasting.

Question 6: Load a time series dataset (e.g., AirPassengers), plot the original series, and decompose it into trend, seasonality, and residual components

```
import pandas as pd
import matplotlib.pyplot as plt
from statsmodels.tsa.seasonal import seasonal_decompose

# Load dataset
df = pd.read_csv("AirPassengers.csv")

# Convert Month column to datetime
df['Month'] = pd.to_datetime(df['Month'])

# Set Month as index
df.set_index('Month', inplace=True)

# Display first few rows
print(df.head())

# Plot original time series
plt.figure(figsize=(10, 5))
plt.plot(df['#Passengers'], color='blue')
plt.title('AirPassengers Time Series')
plt.xlabel('Year')
plt.ylabel('Number of Passengers')
plt.grid(True)
plt.show()
```

```
# Decompose the time series
decomposition = seasonal_decompose(
    df['#Passengers'],
    model='multiplicative',   # AirPassengers usually uses multiplicative decomposition
    period=12                 # Monthly data => 12 months seasonality
)

# Plot Trend, Seasonal and Residual Components
decomposition.plot()
plt.suptitle('Time Series Decomposition of AirPassengers Dataset', fontsize=14)
plt.show()
```

Output -



Question 7: Apply Isolation Forest on a numerical dataset (e.g., NYC Taxi Fare) to detect anomalies. Visualize the anomalies on a 2D scatter plot.

```
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.ensemble import IsolationForest

# Load dataset
df = pd.read_csv("NYC_taxi_fare_data.csv")
```

```
# Display dataset information
print(df.head())
print(df.info())

# Select only numerical columns
numeric_df = df.select_dtypes(include=['int64', 'float64'])

# Handle missing values
numeric_df = numeric_df.dropna()

# Train Isolation Forest
model = IsolationForest(
    contamination=0.02,  # Assume 2% anomalies
    random_state=42
)

# Predict anomalies
numeric_df['Anomaly'] = model.fit_predict(numeric_df)

# Count anomalies
print("\nAnomaly Counts:")
print(numeric_df['Anomaly'].value_counts())

# Select first two numerical features for visualization
x_col = numeric_df.columns[0]
y_col = numeric_df.columns[1]

# Plot anomalies
plt.figure(figsize=(10, 6))

normal = numeric_df[numeric_df['Anomaly'] == 1]
anomaly = numeric_df[numeric_df['Anomaly'] == -1]

plt.scatter(normal[x_col], normal[y_col],
            color='blue', label='Normal Data', alpha=0.5)

plt.scatter(anomaly[x_col], anomaly[y_col],
            color='red', label='Anomaly', alpha=0.8)

plt.title('Isolation Forest Anomaly Detection')
plt.xlabel(x_col)
plt.ylabel(y_col)
plt.legend()
plt.grid(True)

plt.show()
```

Question 8: Train a SARIMA model on the monthly airline passengers dataset. Forecast the next 12 months and visualize the results.

```
import pandas as pd
import matplotlib.pyplot as plt
from statsmodels.tsa.statespace.sarimax import SARIMAX

# Load dataset
df = pd.read_csv("AirPassengers.csv")

# Convert Month column to datetime
df['Month'] = pd.to_datetime(df['Month'])

# Set Month as index
df.set_index('Month', inplace=True)

# Time series data
ts = df['#Passengers']

# Build SARIMA model
model = SARIMAX(
    ts,
    order=(1, 1, 1),          # ARIMA(p,d,q)
    seasonal_order=(1, 1, 1, 12)  # Seasonal(P,D,Q,s)
)

results = model.fit()

# Forecast next 12 months
forecast = results.forecast(steps=12)

# Create future dates
future_dates = pd.date_range(
    start=ts.index[-1] + pd.DateOffset(months=1),
    periods=12,
    freq='MS'
)

forecast_series = pd.Series(forecast.values, index=future_dates)

# Plot original data and forecast
plt.figure(figsize=(12, 6))

plt.plot(ts, label='Actual Passengers')
plt.plot(forecast_series,
         label='Forecast (Next 12 Months)',
         color='red',
```

```
        linewidth=2)

plt.title('SARIMA Forecast of Airline Passengers')
plt.xlabel('Year')
plt.ylabel('Passengers')
plt.legend()
plt.grid(True)

plt.show()

# Display forecast values
print("\nForecast for Next 12 Months:")
print(forecast_series)
```

Output -

```
Forecast for Next 12 Months:
1961-01-01    449.330129
1961-02-01    424.386084
1961-03-01    459.031964
1961-04-01    497.864835
1961-05-01    509.862621
1961-06-01    568.258347
1961-07-01    655.810550
1961-08-01    641.190472
1961-09-01    546.392100
1961-10-01    496.800879
1961-11-01    427.673943
1961-12-01    471.235394
Freq: MS, dtype: float64
```

Question 9: Apply Local Outlier Factor (LOF) on any numerical dataset to detect anomalies and visualize them using matplotlib.

```
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.neighbors import LocalOutlierFactor

# Load dataset
df = pd.read_csv("NYC_taxi_fare_data.csv", low_memory=False)

# Select only numerical columns
numeric_df = df.select_dtypes(include=['number'])

# Remove missing values
numeric_df = numeric_df.dropna()
```

```
# Apply LOF
lof = LocalOutlierFactor(
    n_neighbors=20,
    contamination=0.02
)

# Predict anomalies
numeric_df['Anomaly'] = lof.fit_predict(numeric_df)

# Count anomalies
print("Anomaly Counts:")
print(numeric_df['Anomaly'].value_counts())

# Select first two numerical columns for visualization
x_col = numeric_df.columns[0]
y_col = numeric_df.columns[1]

# Sample data if dataset is large
sample_df = numeric_df.sample(
    min(5000, len(numeric_df)),
    random_state=42
)

normal = sample_df[sample_df['Anomaly'] == 1]
anomaly = sample_df[sample_df['Anomaly'] == -1]

# Plot
plt.figure(figsize=(10,6))

plt.scatter(
    normal[x_col],
    normal[y_col],
    alpha=0.5,
    label='Normal Data'
)

plt.scatter(
    anomaly[x_col],
    anomaly[y_col],
    alpha=0.8,
    label='Outliers'
)
```

```
plt.title('Local Outlier Factor (LOF) Anomaly Detection')
plt.xlabel(x_col)
plt.ylabel(y_col)
plt.legend(loc='upper right')
plt.grid(True)

plt.show()
```

Question 10: You are working as a data scientist for a power grid monitoring company. Your goal is to forecast energy demand and also detect abnormal spikes or drops in real-time consumption data collected every 15 minutes. The dataset includes features like timestamp, region, weather conditions, and energy usage. Explain your real-time data science workflow:

● How would you detect anomalies in this streaming data (Isolation Forest / LOF / DBSCAN)?
● Which time series model would you use for short-term forecasting (ARIMA / SARIMA / SARIMAX)?
● How would you validate and monitor the performance over time?
● How would this solution help business decisions or operations?

1. Anomaly Detection

To detect abnormal spikes or drops in energy consumption, I would use **Isolation Forest** because it works well on large datasets, is fast, and can detect unusual patterns without requiring labeled data. Any unusual energy usage would be marked as an anomaly and reported immediately.

2. Time Series Forecasting

For short-term energy demand forecasting, I would use **SARIMAX** because energy consumption shows seasonal patterns and is affected by external factors such as temperature, weather conditions, and region. SARIMAX can handle both seasonality and external variables.

3. Validation and Monitoring

The model would be validated using historical data and evaluated with metrics such as:

- MAE (Mean Absolute Error)
- RMSE (Root Mean Square Error)
- MAPE (Mean Absolute Percentage Error)

The model's performance would be monitored regularly, and it would be retrained when accuracy decreases.

4. Business Benefits

This solution helps the company:

- Predict future energy demand accurately.
- Detect abnormal power usage in real time.
- Prevent power outages and equipment failures.
- Improve resource planning and reduce operational costs.